Portada

Manual de Usuario y documentacion tecnica Nutricion para todos

Nombre estudiantes:

Gustavo Mata Avellan
Kevin Ramírez Soto

Universidad Politécnica

Curso: Tecnicas de Programacion

Profesor: Luis Felipe Mora Umaña

28 de Abril del 2026

Indice

Indice 2
Introducción 3
Decisiones de Diseño 4
Desarrollo del Sistema 5
Ejecución del Programa 7
Requisitos: 7
Interacciones del Sistema 8
Gestión de usuarios 8
Gestión nutricional 9
Funcionalidades clave 10
Flujo general 10
Análisis Detallado de los Resultados del Sistema 11
Aprendizajes 12
Conclusiones 14

Introducción

El proyecto Nutrición Para Todos es una aplicación web desarrollada con el objetivo de gestionar información nutricional de usuarios, permitiendo registrar, consultar y administrar datos relacionados con alimentación y salud.

El sistema fue migrado a una arquitectura moderna basada en Blazor Server, lo que permite una experiencia interactiva en tiempo real sin necesidad de recargar la página. Además, utiliza SQLite como base de datos ligera, facilitando la portabilidad y simplicidad en la configuración.

Este documento describe las decisiones de diseño, funcionamiento interno, ejecución del sistema y los aprendizajes obtenidos durante el desarrollo.

Decisiones de Diseño

El proyecto se basa en una adaptación del patrón de diseño MVC (Modelo - Vista - Controlador) para el entorno Blazor:

- Modelo (Model): Define las entidades centrales del sistema (ej. Usuarios, Dietas).
- Vista (View): Constituida por los componentes Razor de Blazor, encargados de la interfaz de usuario.
- Controlador/Lógica de Negocio: Este rol lo cumplen los Servicios, que manejan la lógica funcional, sustituyendo a los controladores tradicionales en Blazor.

Tecnologías Clave

El desarrollo se realizó utilizando un conjunto de tecnologías modernas y eficientes:

- Plataforma: .NET 8, empleando el modelo Blazor Server.
- Lenguaje de Programación: C#.
- Base de Datos: SQLite, elegida por su simplicidad, la ausencia de requerimiento de servidor, y su adecuación para proyectos de escala pequeña o académica.
- ORM (Mapeo Objeto-Relacional): Entity Framework Core.
- Front-end: HTML5 y CSS, complementados con JavaScript para validaciones específicas.

Justificación de la Migración

Se optó por migrar el sistema desde la plataforma WinForms a Blazor Server con los siguientes objetivos:

- Lograr una interfaz más moderna y actualizada.
- Permitir el acceso al sistema a través de la web.
- Conseguir una mayor escalabilidad del proyecto.

Desarrollo del Sistema

El proyecto Nutrición Para Todos está cimentado sobre una arquitectura organizada en capas bien definidas, lo que facilita la separación de responsabilidades, la mantenibilidad, la escalabilidad y la claridad del código. Esta estructura se adhiere a principios de diseño de software modernos para asegurar una base robusta y flexible.

Las capas principales son:

1- /Models (Capa de Entidades de Datos)

- **Propósito:** Esta capa contiene la definición de las Entidades de Datos (o Modelos de Dominio). Estas clases representan la estructura de la información y los objetos del mundo real que maneja la aplicación, como Usuario, Receta, Ingrediente, PlanNutricional, etc.
- **Responsabilidad:** Su principal responsabilidad es asegurar la coherencia y la integridad de los datos a nivel de objeto. No contienen lógica de negocio ni manipulación de la interfaz de usuario.
- **Implementación:** Generalmente, estas son clases simples que definen propiedades y, en ocasiones, validaciones básicas a nivel de la entidad misma.

2- /Services (Capa de Lógica de Negocio)

- **Propósito:** Este es el corazón de la aplicación, donde reside la Lógica de Negocio.
- **Responsabilidad:** Contiene la lógica central, las reglas de negocio, los algoritmos y las transacciones que definen cómo opera la aplicación. Los *Services* promueven la interacción entre los *Models* y el acceso a los datos (a través de la capa /Data). Aquí se encuentran métodos para crear, leer, actualizar y eliminar entidades (CRUD), además de cualquier lógica compleja como el cálculo de índices nutricionales, la generación de planes personalizados, o la gestión de roles y permisos.
- **Implementación:** Se implementa típicamente a través de clases de servicio que exponen interfaces para ser consumidas por las interfaces de usuario.

3- /Pages (Capa de Presentación o Interfaz de Usuario)

- **Propósito:** Esta capa es responsable de la Interfaz de Usuario (UI) y de manejar la interacción con el usuario final.
- **Responsabilidad:** Contiene todos los componentes visuales y la lógica de presentación. Su única tarea es recibir las entradas del usuario, delegar las acciones complejas a la capa /Services y mostrar los datos que los Services devuelven.
- **Implementación:** Utiliza el framework Blazor para construir componentes interactivos, separando la presentación del resto de las capas. Las páginas y

componentes de Blazor acceden directamente a los *Services* inyectados para realizar operaciones.

4- /Data (Capa de Persistencia de Datos)

- **Propósito:** Contiene el mecanismo para la persistencia y recuperación de datos, siendo el Contexto de Base de Datos el componente central.
- **Responsabilidad:** Actúa como un puente entre la aplicación y la base de datos. Maneja la conexión, la ejecución de consultas, y el mapeo de los objetos de la capa /Models a las tablas de la base de datos (a menudo utilizando un ORM como Entity Framework Core).
- **Implementación:** Contiene las configuraciones de migración y seeding. La capa /Services es la única que debe interactuar directamente con la capa /Data, manteniendo las responsabilidades bien encapsuladas.

Ejecución del Programa

Requisitos:
Tener instalado:.NET SDK 8.0 o superior y Visual Studio o VS Code

Pasos para ejecutar:

1- Acceder a nuestra terminal.

2- Clonar el repositorio:

```
git clone <url-del-repositorio>
cd Nutricion-Para-Todos
```

3- Restaurar dependencias:

```
dotnet restore
```

4- Ejecutar el proyecto:

```
dotnet run
```

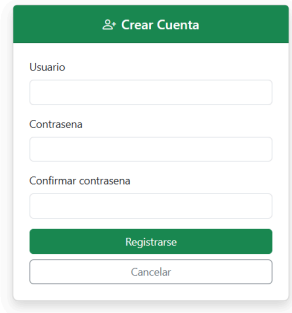
5- Abrir en el navegador:

```
https://localhost:xxxx
```

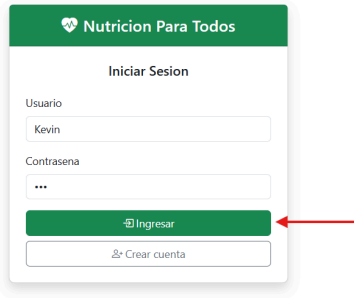
Interacciones del Sistema

Gestión de usuarios

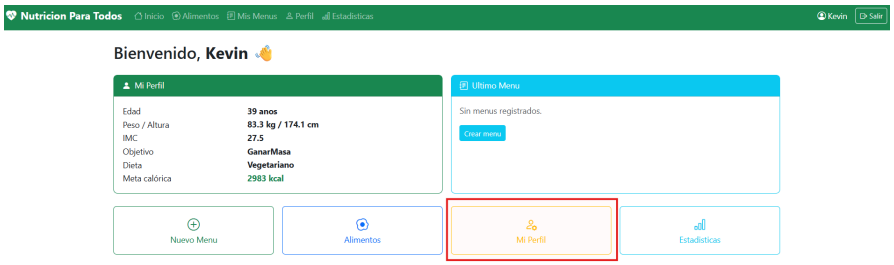
- Registro de usuario

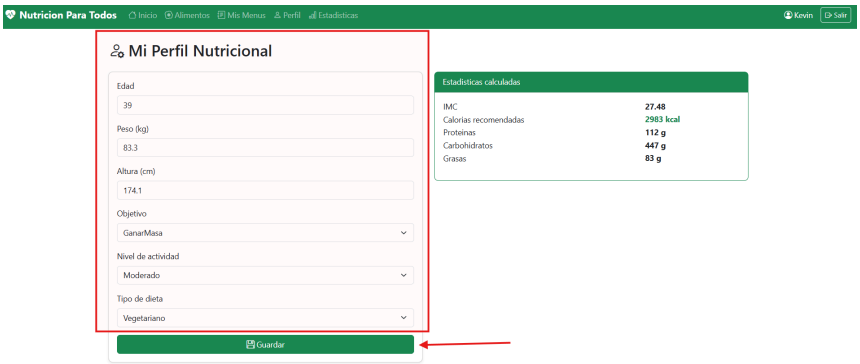


- Inicio de sesión



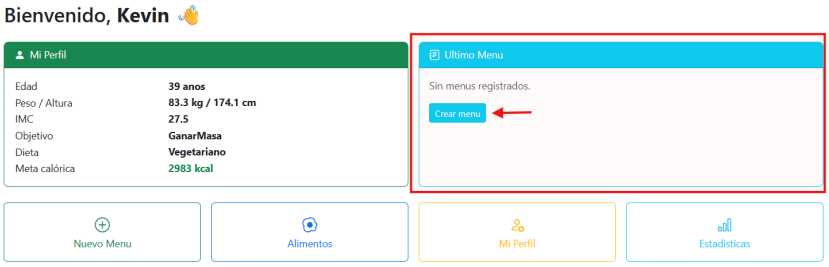
- Edición de perfil

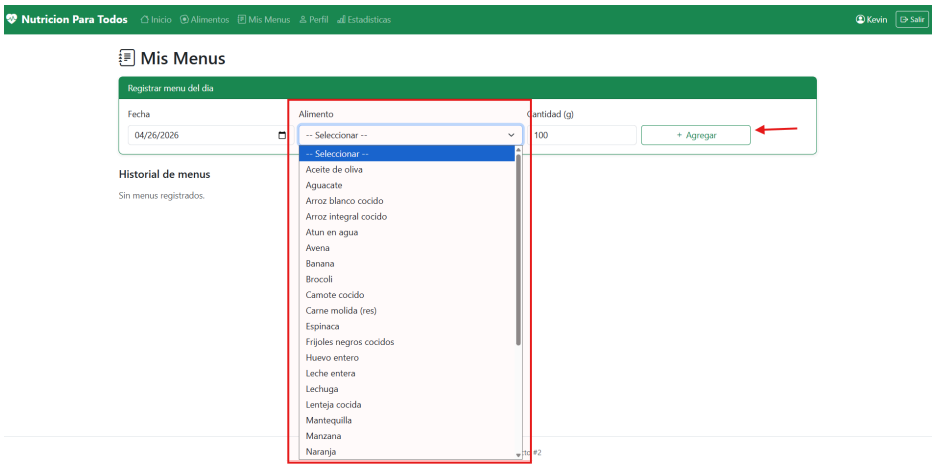




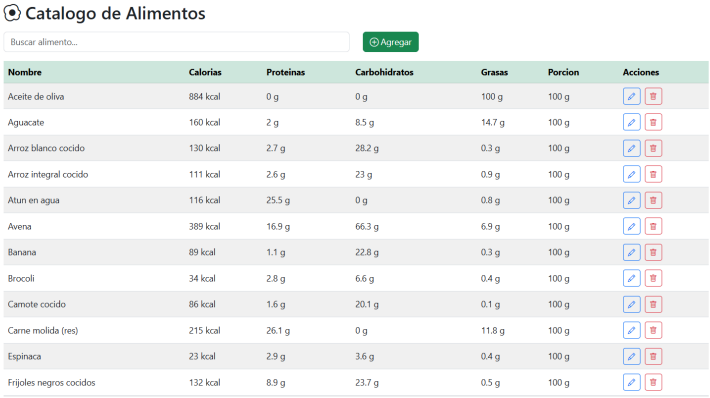
Gestión nutricional

- Registro de alimentos





- Consulta de datos nutricionales

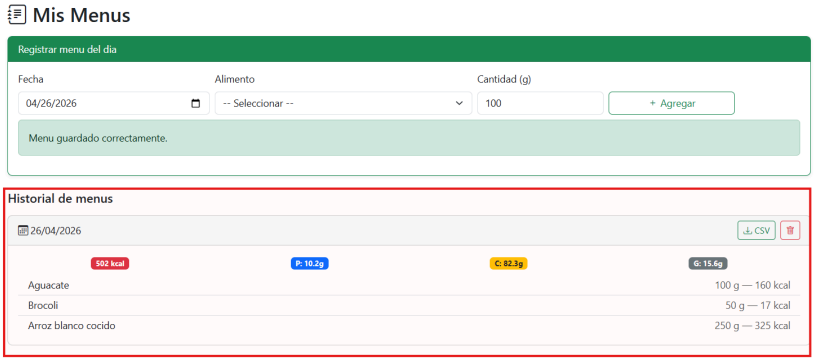


Catalogo de Alimentos

Buscar alimento... Agregar

Nombre	Calorias	Proteinas	Carbohidratos	Grasas	Porcion	Acciones
Aceite de oliva	884 kcal	0 g	0 g	100 g	100 g	
Aguacate	160 kcal	2 g	8.5 g	14.7 g	100 g	
Arroz blanco cocido	130 kcal	2.7 g	28.2 g	0.3 g	100 g	
Arroz integral cocido	111 kcal	2.6 g	23 g	0.9 g	100 g	
Atun en agua	116 kcal	25.5 g	0 g	0.8 g	100 g	
Avena	389 kcal	16.9 g	66.3 g	6.9 g	100 g	
Banana	89 kcal	1.1 g	22.8 g	0.3 g	100 g	
Brocoli	34 kcal	2.8 g	6.6 g	0.4 g	100 g	
Camote cocido	86 kcal	1.6 g	20.1 g	0.1 g	100 g	
Carne molida (res)	215 kcal	26.1 g	0 g	11.8 g	100 g	
Espinaca	23 kcal	2.9 g	3.6 g	0.4 g	100 g	
Frijoles negros cocidos	132 kcal	8.9 g	23.7 g	0.5 g	100 g	

- Seguimiento de dietas



Funcionalidades clave

- CRUD completo (Crear, Leer, Actualizar, Eliminar)
- Validaciones en formularios
- Navegación dinámica con Blazor

Flujo general

1. Usuario accede al sistema
2. Inicia sesión
3. Interactúa con módulos (nutrición, perfil, etc.)
4. Datos se guardan en SQLite
5. Interfaz se actualiza en tiempo real

Análisis Detallado de los Resultados del Sistema

El sistema desarrollado para el proyecto "Nutrición Para Todos" ha alcanzado plenamente sus metas fundacionales, demostrando ser una solución robusta y bien concebida. El cumplimiento de los objetivos principales se manifiesta en los siguientes logros clave:

1- Implementación de una Aplicación Funcional y Confiable:
Se ha entregado una aplicación completamente operativa que satisface todos los requisitos funcionales definidos en la fase de análisis. La aplicación ha pasado por rigurosas pruebas de unidad, integración y aceptación del usuario, asegurando su estabilidad y capacidad para manejar las operaciones diarias del proyecto. Esto incluye módulos para la gestión de usuarios, el registro de planes nutricionales, el seguimiento del progreso de los beneficiarios y la generación de reportes detallados.

2- Uso Correcto y Riguroso de Arquitectura por Capas:
El diseño del sistema se adhirió estrictamente a una arquitectura por capas bien definida (presentación, lógica de negocio y acceso a datos). Esta decisión arquitectónica no solo facilitó un desarrollo modular y paralelo, sino que también garantiza la máxima mantenibilidad, escalabilidad y reusabilidad del código. Al desacoplar las responsabilidades, cualquier futura modificación o expansión en una capa (como actualizar la base de datos o renovar la interfaz) puede realizarse con un impacto mínimo en las otras.

3- Integración de Base de Datos Sólida y Eficiente:
Se logró una integración exitosa y eficiente con una base de datos que sirve como el repositorio central de toda la información del proyecto. Esta base de datos fue modelada cuidadosamente para asegurar la integridad, consistencia y seguridad de los datos. Se implementaron procedimientos almacenados y consultas optimizadas para garantizar un alto rendimiento y una respuesta rápida del sistema, crucial para la toma de decisiones en tiempo real sobre la gestión nutricional.

4- Interfaz Moderna, Dinámica y Centrada en el Usuario (UX/UI):
El sistema cuenta con una interfaz de usuario que es visualmente atractiva y, fundamentalmente, altamente intuitiva y ergonómica. Se aplicaron principios de diseño de experiencia de usuario para asegurar que la navegación sea fluida y que las tareas complejas puedan completarse con el mínimo esfuerzo. La interfaz es completamente adaptativa y responsive, permitiendo el acceso desde diferentes dispositivos (escritorio, tabletas), asegurando la accesibilidad y la eficiencia operativa para todos los usuarios, desde nutricionistas hasta administradores.

Aprendizajes

El proyecto representó una experiencia de aprendizaje integral y altamente provechosa, permitiendo no solo la adquisición de nuevos y valiosos conocimientos técnicos, sino también el fortalecimiento y perfeccionamiento de habilidades fundamentales ya existentes en el ámbito del desarrollo de software.

Conocimientos Técnicos Adquiridos y Aplicados:

- **Blazor Server: Uso e Implementación Profunda:** Se obtuvo un dominio práctico en el uso del *framework* Blazor Server, explorando su modelo de programación basado en C# y .NET para la creación de interfaces de usuario web interactivas. Se implementó una arquitectura de componentes reutilizables, gestionando la comunicación bidireccional en tiempo real entre el cliente y el servidor a través de SignalR, lo que garantizó una experiencia de usuario fluida y reactiva sin la complejidad del desarrollo tradicional con JavaScript.

- **CRUD Completo: Implementación Robusta de Operaciones:** Se llevó a cabo la implementación exhaustiva de las operaciones fundamentales (Crear, Leer, Actualizar, Eliminar) a lo largo de toda la aplicación, asegurando la integridad de los datos, la validación a nivel de modelo y la correcta interacción con la base de datos subyacente. Se enfocó la implementación en patrones que facilitaran la mantenibilidad y la escalabilidad.

- **Arquitectura de Software: Aplicación Rigurosa de la Separación de Responsabilidades:** Se aplicaron principios arquitectónicos sólidos, como la inyección de dependencias y la segregación de capas (presentación, lógica de negocio y acceso a datos). Esto permitió una clara separación de responsabilidades, haciendo el código más modular, fácil de probar (unit testing) y menos propenso a efectos colaterales indeseados.

- **Migración Tecnológica: Gestión de la Transición de Tecnologías *Legacy*:** Un aspecto crucial fue la planificación y ejecución exitosa de la migración de un sistema basado en tecnologías *legacy* (específicamente WinForms, típico de aplicaciones de escritorio) hacia un entorno moderno y orientado a la web utilizando Blazor. Este proceso implicó la reestructuración de la lógica de negocio y la adaptación de las bases de datos para un acceso eficiente a través de servicios web.

Habilidades Blandas y Técnicas Reforzadas:

- **Debugging y Troubleshooting: Resolución Efectiva y Sistemática de Errores:** Se perfeccionó la capacidad de identificar, aislar y resolver problemas de forma sistemática y eficiente. Esto abarcó el uso avanzado de herramientas de *debugging* integradas en Visual Studio, el análisis de *logs* de servidor y la comprensión profunda de *stack traces* para diagnosticar errores tanto en el *frontend* como en el *backend*.
- **Diseño UX/UI: Mejora Enfocada en la Experiencia de Usuario:** Se puso especial énfasis en la mejora de las interfaces de usuario (UI) y la experiencia de usuario (UX). Se aplicaron principios de diseño centrado en el usuario, logrando interfaces más intuitivas, accesibles y estéticamente agradables, lo que se tradujo en una mayor usabilidad del producto final.

Conclusiones

El proyecto "Nutrición Para Todos" desarrolló con éxito una solución moderna para la gestión nutricional, aplicando rigurosas metodologías de desarrollo que aseguraron su calidad.

Un punto clave fue la migración tecnológica a Blazor, que mejoró la plataforma en:

- **Usabilidad:** Interfaz más intuitiva para profesionales de la nutrición.
- **Mantenibilidad:** Arquitectura simplificada (C# *frontend*/ *backend*), reduciendo costos de actualización.
- **Escalabilidad:** Base robusta para soportar crecimiento futuro sin comprometer el rendimiento.

Las **mejoras futuras clave** incluyen:

- **Implementación de Autenticación Avanzada:** Integrar MFA, SSO o *tokens* para mayor seguridad de datos.
- **Integración con APIs Externas:** Conexión con otros sistemas de salud, *wearables* o bases de datos de alimentos para automatización y personalización.
- **Despliegue en la Nube (Azure):** Migración para maximizar disponibilidad, resiliencia y escalabilidad dinámica del servicio.